

Lab Six

User Interface Testing

The difficult part of creating UI tests is discovering the elements in the GUI needed for entering inputs and discovering the elements that will display the outputs. This can be a challenge in communication when creating the tests for a human, but it is a further challenge to create the automated system for checking those elements.

In Lab Five, you used Postman to create tests for the API or the back-end “reception desk” which provides services and information the front end will display.

In this lab, we will move on to creating tests for the Front End.

This lab will the same class roster system, which is built on including CRUD operations for Students and Courses. The API will support creating and updating a Student (with name, Major, and GPA) and a Course (with name, size, room, instructor, and roster—a list of Students). There will be functions for adding a student to a course, and removing a student from a course, as well as updating all the fields.

A few Business Rules to begin with: GPA should be between 0 and 4.0. Names should not be longer than 255 characters. The number of students added to a class should not exceed the size of the class. Note that error handling on the frontend has not been implemented, but tests should still test for these things.

So, you will need to design and create tests for creating, updating, retrieving, and deleting students and courses, as well as enforcing the various format and business rules for those entities. You may be able to re-use most of your basic TCIs from the previous lab—but you will need to modify the test cases to account for the front-end interface. Then design the tests using Selenium.

Here are the relevant element IDs:

navigation links to Course list page and student list page:

Page: any

Link: “nav-course-list-link”

Link: “nav-student-list-link”

Page: StudentReg or CourseList.tsx in the code. URL: IP:Port (root for the Vite server)

div: “new-course-fields”

```

input: "new-course-name"
input: "new-course-instructor"
input: "new-course-max-size"
input: "new-course-room"
table: "course-list-table"
  tr: `{course-row-${course.id}}` // any id will be "course-row-" + the id number of the
    course
  td: `{course-name-${course.id}}`
  td: `{course-instructor-${course.id}}`
  td: `{course-max-size-${course.id}}`
  td: `{course-room-${course.id}}`
  td: `{course-roster-${course.id}}`
  button: "delete-course-button"
  button: "edit-course-button"
  select: "select-student"
  button: "add-student-button"
    input: "edit-course-name"
    input: "edit-course-instructor"
    input: "edit-course-max-size"
    input: "edit-course-room"
    select: "remove-student-select"
    button: "remove-student-button"
    button: "edit-course-save-button"
    button: "edit-course-cancel-button"

```

Page: Student Reg or StudentList.tsx in code; URL: IP:Port/students

```

div: "new-student-fields"
  input: "new-student-name"
  input: "new-student-major"
  input: "new-student-gpa"
table: "student-list-table"
  tr: `{student-row-${student.id}}` // any id will be "student-row-" + the id number of
    the student
  td: `{student-name-${student.id}}`
  td: `{student-major-${student.id}}`
  td: `{student-gpa-${student.id}}`
  button: "delete-student-button"
  button: "edit-student-button"
    input: "edit-student-name"
    input: "edit-student-major"
    input: "edit-student-gpa"

```

button: "edit-student-save-button"
button: "edit-student-cancel-button"

Note that you will not be able to add any students to courses until you have created students. So, the order of tests should probably start with adding and editing students. Remember to have a test case document attached to the project.

There are many ways to run the project:

After all your tests have been completed, you should be able to run the Selenium tests with a mere `mvn clean verify` command from terminal window.

To run both the backend and the frontend servers at the same time:

1. Navigate to the project with the POM file in it in a terminal window.
2. Type: `mvn spring-boot: run`
This should start the Spring Boot Tomcat server, with the backend endpoints and the H2 database on port `localhost:8080`
3. Open a new terminal window.
4. Navigate to the project with the POM file, and then go into the frontend directory.
5. Type: `npm run dev`
This should start the front end Vite server on port `localhost:5173`